AdaPart-GNN: Scale-Adaptive Graph Partitioning with Boundary Prompting for Scalable GNNs

Surendra Kumar Raut
Department of CSE
IIT Bhilai, India
surendrakr@iitbhilai.ac.in

Kishan Tamboli
Department of CSE
IIT Bhilai, India
kishant@iitbhilai.ac.in

Vishwesh Jatala
Department of CSE
IIT Bhilai, India
vishwesh@iitbhilai.ac.in

***Abstract*—Nowadays, large-scale graph processing is performed using Graph Neural Networks (GNNs) to find patterns in graph-structured data; however, it faces severe memory and computational bottlenecks during training. As a remedy, partition-based training reduces GPU memory by dividing graphs into smaller subgraphs and training them sequentially, but discarding inter-partition edges causes information loss and accuracy degradation. Moreover, fixed partition configurations may lead to inefficient GPU memory utilization.**

To overcome these challenges, we introduce AdaPart-GNN, an efficient and boundary-preserving framework for large-scale GNN training. AdaPart-GNN first determines a scale-adaptive partition count (K^*) based on graph characteristics and GPU memory constraints. Second, it employs Jaccard-Eigenvector weighting to identify and preserve structurally dense communities during graph partitioning. Finally, a lightweight Boundary Eigen-Prompt module injects topological prompts into boundary nodes, restoring severed cross-partition signals without communication. Experiments on five benchmark datasets show up to 31.2× speedup (16.5× average) over DGL GraphSAGE, up to 64.4% peak VRAM reduction, +11.7% accuracy on Ogbn-arxiv, and +13.2% Micro-F1 on Yelp.

I. Introduction

Graph Neural Networks (GNNs) are the foundational paradigm for machine learning on relational structures. However, recursive multi-hop message passing triggers the well-known *neighborhood explosion* problem: receptive fields expand exponentially with network depth, resulting in prohibitive computation and memory footprints that exceed commodity GPU memory.

To scale GNN training, two primary strategies exist: stochastic neighbor sampling (e.g., GraphSAGE [1]), which bounds neighborhood sizes but incurs redundant data movement and high sampling latency; and Subgraph sampling (e.g., Cluster-GCN [2]), which divides the graph into subgraphs and trains mini-batches strictly within partitions.

While partitioning bounds peak VRAM to partition size and avoids redundant transfers, it suffers from the *boundary cut-edge dilemma*: severing cross-partition edges isolates boundary nodes from essential neighborhood messages, degrading representation quality on relational graphs. Furthermore, rigid, topology-oblivious partition counts (K) cause either GPU Out-Of-Memory (OOM) exceptions or excessive edge cuts.

To overcome these challenges, we propose **AdaPart-GNN**, an efficient, scale-adaptive, and boundary-preserving GNN training framework. Our contributions are three-fold:

- **Scale-Adaptive Partitioning**: An analytical heuristic (K^*) that determines the partition count based on graph scale, structural density, and GPU memory limits.
- **Jaccard-Eigenvector Edge Weighting**: A hybrid metric combining local cohesion and eigenvector centrality to guide METIS toward preserving vital dense communities.
- **Boundary EigenPrompt Bridging**: A parameter-efficient prompt module restoring cross-partition topological awareness for cut nodes without inter-subgraph communication.

Across five diverse benchmarks, AdaPart-GNN achieves an average speedup of **16.5**× over DGL (GraphSAGE), slashes peak GPU memory by up to **64.4%**, and boosts test accuracy by up to **+11.7%** over unweighted partition baselines.

II. Approach

AdaPart-GNN addresses scalability and accuracy trade-offs in partition-based GNNs via three synergistic stages: (1) scale-adaptive partition determination, (2) Jaccard-eigenvector edge weighting, and (3) boundary prompt bridging (Figure 1).

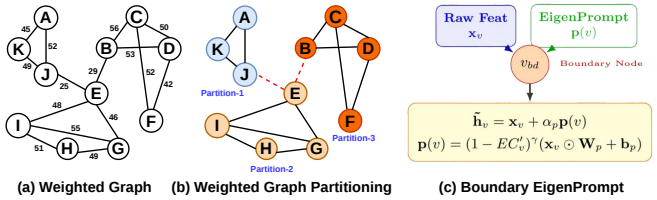
1) Scale-Adaptive Partition Sizing (K^*): Fixed partition counts cause Out-Of-Memory (OOM) failures on massive graphs or severe community disruption on smaller graphs. To balance hardware limits with topological fidelity, we formulate a scale- and density-aware sizing heuristic:

$$K^* = \left\lceil \frac{|V|}{N_{\text{target}}} \cdot \left(1 + \beta \log_{10}\left(1 + \frac{2|E|}{|V|}\right)\right)\right\rceil \tag{1}$$

where $N_{\text{target}} \approx \frac{M_{\text{gpu}}}{\text{Mem}_{\text{node}}} \approx 35{,}000$–$40{,}000$ is the hardware-budgeted node capacity per subgraph (≤ 1.5 GB GPU allocation, accommodating feature tensors and SpMM buffers), setting $\frac{|V|}{N_{\text{target}}}$ as the baseline partition count. The average degree $\frac{2|E|}{|V|}$ measures structural density, penalizing edge-heavy graphs where message passing inflates activation memory, while $\log_{10}$ dampens power-law skews ($\beta \approx 0.35$). Intuitively, Equation (1) scales K^* on massive graphs ($K^* = 69$ on Products) to avoid OOMs while preserving relational structure on smaller graphs ($K^* = 13$ on Arxiv).

2) Jaccard-Eigenvector Edge Weighting: Standard METIS partitioning minimizes unweighted cuts, severing vital dense communities. To retain high-centrality pathways, we formulate a composite similarity combining neighborhood overlap (Jaccard) with global influence (eigenvector centrality):

$$S_{uv} = J_{uv}^{\alpha} \cdot \left(\sqrt{EC'_u \cdot EC'_v}\right)^{1-\alpha}, \quad w_{uv} = 1 + \lfloor K_{\text{scale}} \cdot S_{uv} \rceil \tag{2}$$



(a) Weighted Graph (b) Weighted Graph Partitioning (c) Boundary EigenPrompt

Fig. 1: AdaPart-GNN framework: (a) Weighted Graph, (b) Weighted Graph Partitioning with severed cuts, and (c) Boundary EigenPrompt bridging.

where $J_{uv} = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$ captures community cohesion, and $EC'_v = \frac{EC_v}{\max_k EC_k} \in [0,1]$ is the normalized eigenvector centrality. We empirically set $\alpha = 0.7$, balancing local cohesion with global topology while keeping $S_{uv} \in [0,1]$. To satisfy METIS integer requirements, $w_{uv} = 1 + \text{round}(K_{\text{scale}} S_{uv})$ ($K_{\text{scale}} = 100$) maps weights to $[1, 101]$, guiding METIS to preserve cohesive partitions and cutting boundary loss by up to 8.1% on Reddit.

3) Boundary EigenPrompt Bridging: When a graph is partitioned, boundary nodes lose connections to neighbors in other partitions. While transferring cross-partition features restores this context, it causes heavy communication overhead. To solve this with zero communication, we inject a lightweight *Boundary EigenPrompt* $\mathbf{p}(v)$ directly into node features $\mathbf{x}_v$:

$$\tilde{\mathbf{h}}_v = \mathbf{x}_v + \alpha_p \cdot \mathbf{p}(v), \quad \mathbf{p}(v) = (1 - EC'_v)^\gamma \cdot (\mathbf{x}_v \odot \mathbf{W}_p + \mathbf{b}_p) \quad (3)$$

Here, $\mathbf{W}_p, \mathbf{b}_p \in \mathbb{R}^D$ are learnable vectors shared across subgraphs to provide global context, and α_p is a scaling factor ensuring the prompt supplements rather than overpowers original node features $\mathbf{x}_v$. The term $(1 - EC'_v)^\gamma$ adaptively modulates prompt strength based on node centrality: isolated boundary nodes ($EC'_v \approx 0$) receive full prompt compensation, while well-connected interior hubs ($EC'_v \approx 1$) receive near-zero prompt ($(1 - EC'_v)^\gamma \to 0$) to protect intact features. Setting $\gamma = 1.5$ smoothly suppresses prompt noise on moderately connected nodes. Since only $\mathbf{W}_p$ and $\mathbf{b}_p$ are trained, the entire prompt module and its optimizer states consume < 6 MB of GPU memory, restoring boundary accuracy with zero runtime communication.

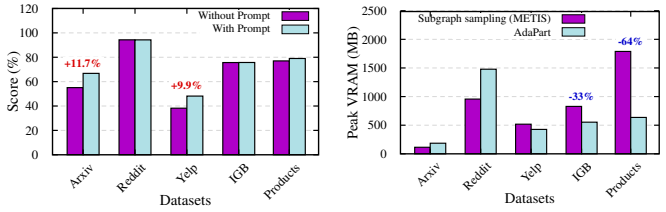
III. Experimental Evaluation

We performed our experiments on a single NVIDIA RTX A6000 GPU (48 GB), Intel Xeon Gold 5218 CPU (16 cores), and 512 GB RAM with PyTorch 2.1 and DGL 2.4.0. Evaluated datasets, properties, and adaptive partitions (K^*) are summarized in Table I. Results use a 3-layer GraphSAGE (dim 256, LayerNorm, Adam lr 0.001, 100 epochs).

Comparison with State-of-the-Art: As reported in Table I, AdaPart-GNN outperforms state-of-the-art stochastic sampling (DGL GraphSAGE [1]) and Subgraph sampling (METIS [3]), reaching up to **31.2**× speedup on Yelp and **25.0**× on IGB-small (**16.5**× average) over DGL GraphSAGE due to: (1) eliminating redundant neighbor sampling and transfers, and (2) scale-adaptive sizing (K^*), optimizing GPU residency and SpMM throughput vs. Subgraph sampling (METIS).

TABLE I: Comparison with DGL (GraphSAGE) and Subgraph sampling (METIS) (M: million; Micro-F1 in brackets for Yelp).

Dataset	$\lvert V\rvert$	$\lvert E\rvert$	K^*	*Metrics*	*DGL (SAGE)*	*Subgraph sampling (METIS)*	*AdaPart-GNN*
Ogbn-arxiv	0.17M	1.17M	13	Train Time (s)	127.11	17.48 (7.3×)	**11.42 (11.1×)**
				Accuracy	0.56	0.55	**0.67 (+11%)**
Reddit	0.23M	114.6M	17	Train Time (s)	633.18	**60.27 (10.5×)**	61.19 (10.4×)
				Accuracy	**0.96**	0.94	0.94
Yelp	0.72M	13.95M	26	Train Time (s)	1687.25	59.10 (28.6×)	**54.15 (31.2×)**
				Acc (Micro-F1)	0.92 (0.41)	0.92 (0.35)	**0.93 (0.48)**
IGB-small	1.00M	12.07M	32	Train Time (s)	4259.40	**170.01 (25.1×)**	170.58 (25.0×)
				Accuracy	0.74	0.75	**0.76**
Ogbn-products	2.45M	61.86M	69	Train Time (s)	717.15	149.05 (4.8×)	**140.39 (5.1×)**
				Accuracy	0.78	0.77	**0.79**



(a) EigenPrompt Ablation (b) Peak VRAM Footprint

Fig. 2: (a) EigenPrompt ablation (Accuracy; Micro-F1 for Yelp), and (b) Peak VRAM vs. Subgraph sampling (METIS) ($K = 20$).

Peak Memory Scalability: As shown in Figure 2(b), adaptive sizing (K^*) cuts peak VRAM by **64.4%** on Ogbn-products (1,788.7 MB to **635.9 MB**) and **33.3%** on IGB-small vs. Subgraph sampling (METIS) ($K = 20$). For smaller graphs, K^* avoids over-fragmentation, fitting budget GPUs (≤ 1.5 GB) while preserving topology.

Effect of Boundary EigenPrompt (Ablation): To isolate Boundary EigenPrompt's impact, Figure 2(a) compares AdaPart-GNN (With Prompt) against unprompted training (Without Prompt, $\mathbf{p}(v) = \mathbf{0}$). Prompting recovers severed context on cut-sensitive graphs, boosting accuracy by **+11.7%** on Ogbn-arxiv (to **66.78%**) and Micro-F1 by **+9.9%** on Yelp (to **48.15%**). On dense graphs (Reddit), the boundary gate $(1 - EC'_v)^\gamma$ attenuates prompting with < 6 MB memory overhead and zero latency.

IV. Conclusion

We proposed AdaPart-GNN, an efficient partitioning framework for scalable GNN training. By combining adaptive partition sizing (K^*), edge weighting, and Boundary EigenPrompt bridging, AdaPart-GNN eliminates cross-partition communication while recovering structural context. It achieves up to **31.2**× speedup over DGL (GraphSAGE) and **64.4%** peak VRAM reduction, with consistent accuracy gains (+11.7% on Arxiv). Future work explores distributed multi-GPU scaling.

References

[1] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *NeurIPS*, pp. 1024–1034, 2017.

[2] W. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C. Hsieh, "Cluster-GCN: An efficient algorithm for training deep and large graph convolutional networks," in *KDD*, pp. 257–266, 2019.

[3] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, vol. 20, no. 1, pp. 359–392, 1998.